VICTOR. A

OPEN CLASSROOM

PROJET 10

RÉALISEZ UNE APPLICATION DE RECOMMANDATION DE CONTENU

CONTEXTE ET OBJECTIFS DU PROJET

- **Contexte :** MyContent est une start-up visant à encourager la lecture grâce à la recommandation de contenus. Ce projet consiste à concevoir un premier MVP de système de recommandation à partir de données d'interactions utilisateurs.

- **Objectif :** Développer un système de recommandation capable de proposer, pour un utilisateur donné, cinq articles pertinents, dans une logique de MVP simple, fonctionnelle et évolutive.

MISE EN PLACE

Tableau 1 — Données utilisées (aperçu & structure)

Type de données	Champ	Description	Exemple
Articles	article_id	Identifiant de l'article	0
Articles	category_id	Catégorie de l'article	0
Articles	created_at_ts	Date de publication (timestamp)	1513144419000
Articles	publisher_id	Identifiant de l'éditeur	0
Articles	words_count	Nombre de mots de l'article	168
Clics	user_id	Identifiant utilisateur	0
Clics	session_id	Identifiant de session	0
Clics	click_article_id	Article cliqué	157541
Clics	click_timestamp	Moment du clic (timestamp)	1506826828020

Tableau 2 — Volumétrie du jeu de données

Indicateur	Valeur
Nombre d'articles	364 047
Nombre d'utilisateurs	707
Nombre total de clics	1 883

L'exploration initiale des données montre un catalogue d'articles très volumineux comparé au nombre limité d'interactions utilisateurs. Le jeu de données est donc fortement creux, avec peu de clics par utilisateur. Ce contexte oriente le projet vers une approche de recommandation simple et robuste, adaptée à une logique de MVP et capable de fonctionner malgré une faible densité d'interactions.

MISE EN PLACE

Embeddings et couverture des interactions

Élément	Valeur	Interprétation technique	Impact sur le système
Dimension des embeddings	(364 047, 250)	Représentation vectorielle dense de chaque article	Base du calcul de similarité
article_id min / max (clics)	2 137 / 363 291	Les identifiants cliqués sont inclus dans le catalogue	Pas de perte d'information
Couverture des clics	100 %	Tous les articles cliqués disposent d'un embedding	Recommandations possibles pour chaque interaction
Cohérence embeddings / clics	Totale	Alignement parfait entre données de contenu et d'usage	Simplifie l'architecture et le pipeline

L'analyse des embeddings montre une représentation vectorielle complète du catalogue d'articles, avec une dimension fixe de 250 caractéristiques par contenu. La couverture totale des clics par les embeddings garantit que chaque interaction utilisateur peut être exploitée sans perte d'information. Cette cohérence entre données de contenu et données d'usage simplifie le pipeline de recommandation et sécurise la mise en œuvre du calcul de similarité dans le cadre du MVP.

MISE EN PLACE

Articles les plus populaires (Top 5)

Rang	article_id	Nombre de clics	Rôle dans le système
1	119592	Plus élevé	Référence de popularité globale
2	96663	Élevé	Contenu fréquemment consulté
3	108854	Élevé	Article attractif pour les utilisateurs
4	284847	Modéré	Contenu populaire mais moins dominant
5	235840	Modéré	Complément dans le top popularité

Une première baseline de recommandation repose sur la popularité globale des articles, calculée à partir du nombre total de clics observés. Cette approche permet de classer les contenus et de recommander les articles les plus fréquemment consultés. Elle constitue une solution simple et robuste, garantissant des recommandations même en l'absence d'historique utilisateur, notamment dans les situations de *cold start*.

FONCTION RECO

Fonction de recommandation — principes techniques

Composant	Implémentation (code)	Principe algorithmique	Justification
Historique utilisateur	`clicks[clicks["user_id"] == user_id]`	Exploitation des interactions passées	Personnalisation basée sur l'usage réel
Gestion du *cold start*	`if len(hist) == 0: return top_popular[:k]`	Recommandation par popularité	Robustesse en absence de données utilisateur
Profil utilisateur	`u = X[hist].mean(axis=0)`	Agrégation des embeddings cliqués	Représentation vectorielle des intérêts
Normalisation du profil utilisateur	`u = u / max(np.linalg.norm(u), eps)`	Normalisation L2 avec seuil de protection numérique	Assure une norme unitaire du vecteur utilisateur et stabilise le calcul de la similarité cosinus
Calcul de similarité	`scores = X @ u`	Similarité cosinus via produit scalaire	Mesure de proximité sémantique
Réduction du coût de calcul	`np.argpartition(-scores, pool)`	Sélection d'un sous-ensemble de candidats	Optimisation des performances
Filtrage des articles	`aid not in hist_set`	Exclusion des articles déjà consultés	Évite la redondance dans les recommandations
Complément par popularité	Ajout depuis `top_popular`	Fallback par popularité	Garantie de k recommandations

La fonction de recommandation s'appuie sur une approche basée sur le contenu, enrichie par une stratégie de repli par popularité. L'historique de clics permet de construire un profil utilisateur représentatif de ses intérêts, tandis que la popularité garantit des recommandations pertinentes en situation de *cold start*. Cette combinaison offre une solution simple, efficace et parfaitement adaptée aux contraintes d'un MVP.

FONCTION RECO

Démonstration des recommandations

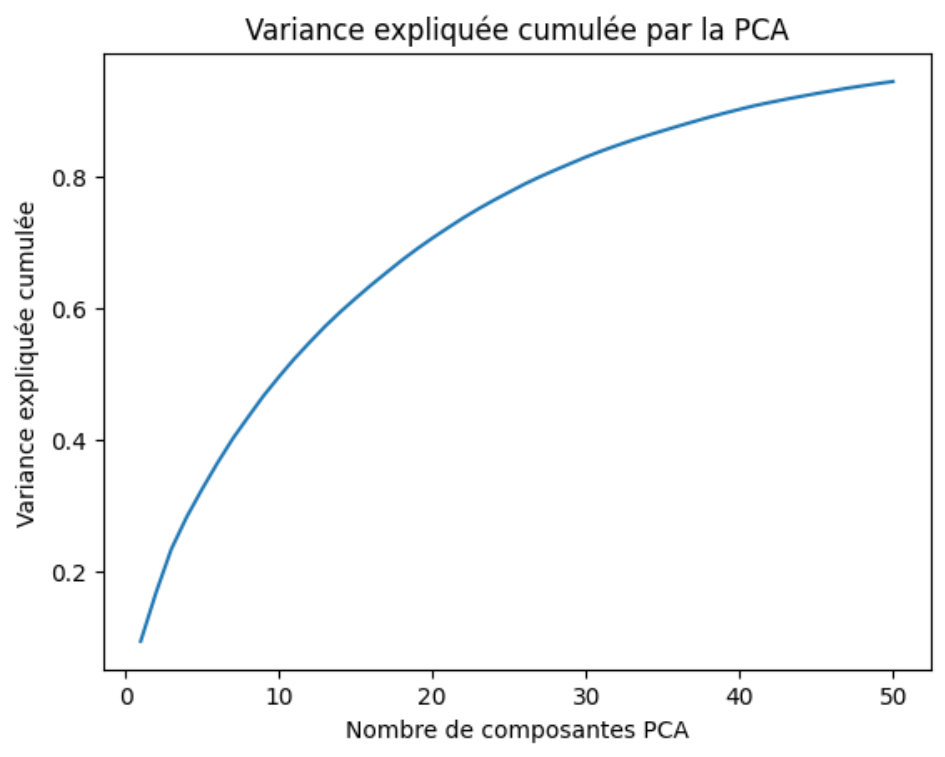
Utilisateur	Historique disponible	Articles recommandés (Top 5)
User 0	Oui	157519, 162856, 159495, 157944, 156690
User 1	Oui	95797, 246739, 90583, 236005, 231234
User 2	Oui	25856, 30847, 23608, 30693, 32503
Nouvel utilisateur	Non (*cold start*)	119592, 96663, 108854, 284847, 235840

Les résultats montrent que le système est capable de générer des recommandations personnalisées distinctes selon les utilisateurs, à partir de leurs historiques de clics. Pour un nouvel utilisateur sans historique, la stratégie de popularité permet de proposer immédiatement des contenus pertinents.

ACP 250 > 50

Réduction de dimension par ACP (PCA)

Élément	Valeur	Interprétation
Dimension initiale des embeddings	250	Représentation vectorielle complète des articles
Dimension après ACP	50	Réduction significative de la dimension
Nombre d'articles	364 047	Aucune perte d'articles lors de la transformation
Variance expliquée cumulée	94,53 %	Conservation de l'essentiel de l'information



L'application de l'ACP permet de réduire fortement la dimension des embeddings, passant de 250 à 50 composantes tout en conservant plus de 94 % de la variance totale. Cette réduction diminue l'empreinte mémoire et le coût de calcul, tout en préservant la structure sémantique des articles.

ACP 250 > 50

Démonstration des recommandations — Version avec ACP

Utilisateur	Historique disponible	Articles recommandés (Top 5)
User 0	Oui	157519, 157944, 159495, 156690, 162857
User 1	Oui	95797, 90583, 236005, 231234, 37095
User 2	Oui	25856, 30693, 32503, 31139, 31325
Nouvel utilisateur	Non (*cold start*)	119592, 96663, 108854, 284847, 235840

Comparaison des recommandations — Avant / Après ACP (Utilisateur 0)

Version du système	Articles recommandés (Top 5)
Avant ACP	157519, 162856, 159495, 157944, 156690
Après ACP	157519, 157944, 159495, 156690, 162857

Les résultats obtenus avec l'intégration de l'ACP montrent que le système conserve des recommandations personnalisées cohérentes tout en bénéficiant d'une réduction significative de la dimension des embeddings. La comparaison avant et après ACP met en évidence une grande stabilité des recommandations, avec seulement des variations mineures, confirmant que l'optimisation n'entraîne pas de dégradation perceptible de la qualité du système.

SYNTHESE EXPLORATION

Synthèse — Exploration et conception de la recommandation

Axe	Constats clés	Choix retenus
Objectif	Recommander 5 articles par utilisateur	Système simple orienté MVP
Données	Catalogue volumineux, peu d'interactions	Approche robuste face à la sparsité
Baseline	Popularité des articles	Gestion du *cold start*
Personnalisation	Historique utilisateur exploitable	Recommandation basée sur le contenu
Représentation	Embeddings de dimension 250	Similarité cosinus
Optimisation	Contraintes de calcul	ACP (50 dimensions, 94 % de variance)
Résultat	Recommandations stables	Solution explicable et déployable

DÉPLOIEMENT

Fichier	Composant	Type	Responsabilité technique	Entrées	Sorties
`recommender.py`	`Recommender`	Classe métier	Implémente la logique centrale de recommandation (contenu + popularité)	`user_id`, `clicks`, `k`, `pool`	Liste d'`article_id` recommandés
`recommender.py`	`recommend()`	Méthode	Génère des recommandations personnalisées avec gestion du *cold start*	Historique utilisateur, embeddings	Top-k recommandations
`recommender.py`	`_l2_normalize()`	Fonction utilitaire	Normalise les embeddings pour la similarité cosinus	Matrice d'embeddings	Embeddings normalisés
`recommender.py`	`build_top_popular()`	Fonction utilitaire	Calcule la popularité globale des articles	Données de clics	Liste d'articles populaires
`recommender.py`	`load_recommender()`	Fonction de chargement	Initialise le système (données, embeddings, popularité)	Fichiers CSV / joblib	Objet `Recommender`, DataFrame clics
`run_local_test.py`	Script principal	Script d'exécution	Lance des tests fonctionnels en local sans interface	Chemin des données	Affichage des recommandations
`run_local_test.py`	Boucle de test	Logique applicative	Teste plusieurs profils utilisateurs (avec / sans historique)	Liste d'`user_id`	Résultats console
`run_local_test.py`	Appel `recommend()`	Appel métier	Vérifie le bon fonctionnement du moteur de recommandation	`user_id`, `k`	Recommandations générées

La structuration du système de recommandation en modules Python distincts permet de séparer clairement la logique métier, les fonctions utilitaires et les scripts d'exécution. Cette organisation facilite les tests locaux, la maintenance et l'évolutivité du code. Elle constitue une base solide pour un déploiement en environnement serverless.

DÉPLOIEMENT


```
{
  "user_id" : 5
  "k" : 5
  "mode" : "content_based_pca50"
  "recommendations" : [
    0 : 274999
    1 : 346131
    2 : 345405
    3 : 203780
    4 : 105921
  ]
}
```

```
{
  "user_id" : 33
  "k" : 5
  "mode" : "content_based_pca50"
  "recommendations" : [
    0 : 108648
    1 : 108942
    2 : 337373
    3 : 31484
    4 : 30178
  ]
}
```

DÉPLOIEMENT STREAMLIT

Composant	Type	Responsabilité technique	Entrées	Sorties
Configuration générale	Paramétrage	Définition du titre, du layout et de l'URL de l'API Azure	URL API, chemins locaux	Configuration de l'UI
`load_data()`	Fonction	Chargement des données locales et calcul de la baseline	CSV articles, CSV clics	DataFrames, user_ids, top_popular
Cache Streamlit	Mécanisme	Mise en cache des données pour éviter les rechargements inutiles	Données locales	Performance et fluidité UI
Sélecteurs utilisateur	Interface	Sélection de l'utilisateur et du nombre de recommandations	`user_id`, `k`	Paramètres de requête
Appel API Azure	Requête HTTP	Appel du moteur de recommandation distant	`user_id`, `k`	Réponse JSON
Gestion des erreurs	Contrôle	Vérification du statut HTTP et des retours API	Code HTTP	Messages d'erreur UI
Affichage des résultats	Visualisation	Affichage des recommandations enrichies	`article_id` recommandés	Tableau interactif
Baseline popularité	Comparaison	Affichage d'une recommandation de référence	Articles populaires	Tableau comparatif

L'application Streamlit joue le rôle de client léger dédié à la démonstration du système de recommandation. Elle se contente de consommer l'API Azure Functions et d'afficher les résultats, sans embarquer de logique de calcul côté interface. Cette séparation garantit une architecture claire, maintenable et cohérente.

DÉPLOIEMENT STREAMLIT

MyContent — MVP de recommandation d'articles

Cette application Streamlit consomme une API Azure Functions exposant un moteur de recommandation d'articles. Aucun calcul de machine learning n'est effectué côté interface.

Identifiant utilisateur

5

Nombre d'articles recommandés

5

Afficher la réponse JSON brute (debug)

Générer les recommandations

Résultats pour l'utilisateur 5 (mode : content_based_pca50, latence : 0.38s)

	article_id	category_id	created_at_ts	publisher_id	words_count
0	274999	404	1472263201000	0	376
1	346131	440	1512370377000	0	795
2	345405	440	1512371313000	0	260
3	203780	328	1412608053000	0	171
4	105921	228	1492033465000	0	182

AZURE FUNCTION

Slide 1 — Application Streamlit (client de démonstration)

Composant	Rôle	Implémentation	Résultat
Configuration	Paramétrage global de l'application	URL de l'API Azure, layout Streamlit	Application prête à l'usage
Chargement des données	Accès local aux métadonnées et baseline	CSV articles et clics avec cache Streamlit	Données rapides et stables
Sélection utilisateur	Choix du profil à tester	Liste d'`user_id` issue des clics	Paramétrage simple de la démo
Paramétrage des recommandations	Ajustement du nombre de résultats	Slider `k` (1 à 10)	Démonstration flexible
Appel du moteur	Consommation du service distant	Requête HTTP vers Azure Functions	Recommandations calculées côté serveur
Affichage des résultats	Visualisation des sorties	Tableaux enrichis avec métadonnées	Lecture claire des recommandations
Comparaison baseline	Mise en perspective des résultats	Top popularité local	Comparaison immédiate

AZURE FUNCTION

Slide 2 — API Azure Functions (moteur de recommandation)

Élément	Description	Justification
Type de service	Azure Functions (HTTP Trigger)	Architecture serverless adaptée au MVP
Point d'entrée	`/api/recommend`	Interface simple et standardisée
Paramètres	`user_id` obligatoire, `k` optionnel	Contrôle direct du comportement
Logique embarquée	Recommandation par contenu + fallback popularité	Gestion native du *cold start*
Optimisation	Embeddings PCA + cache mémoire	Réduction du coût et de la latence
Mode retourné	`content_based_pca50` ou `cold_start_popularity`	Transparence du fonctionnement
Format de réponse	JSON structuré	Consommation facile par tout client
Rôle	Exposition du moteur ML	Séparation stricte UI / logique métier

AZURE FUNCTION

Slide 3 — Déploiement Azure et intégration

Élément	Choix retenu	Intérêt
Déploiement	SDK Azure Functions via VS Code	Mise en production simple et maîtrisée
Langage	Python	Continuité avec le développement ML
Dépendances	`requirements.txt`	Environnement reproductible
Données lourdes	Azure Blob Storage	Séparation code / données
Configuration	Variables d'environnement	Sécurité et flexibilité
Scalabilité	Serverless	Adapté à un usage MVP
Intégration	Streamlit → API Azure	Architecture orientée services

CONCLUSION

Synthèse technique & perspectives — Projet 10 (Système de recommandation)

Axe	Élément analysé	Mise en œuvre technique	Résultat observé	Lecture / enseignement
Synthèse	Données & tâche	Interactions utilisateurs–articles, embeddings de contenu, top popularité	Jeu de données exploitable malgré forte sparsité	Problème réaliste nécessitant des approches robustes
Synthèse	Stratégie de recommandation	Contenu + fallback popularité (*cold start*)	Recommandations systématiques et personnalisées	Bon compromis simplicité / efficacité pour un MVP
Synthèse	Représentation des articles	Embeddings vectoriels normalisés (cosine)	Similarités cohérentes entre contenus	Base solide pour la recommandation basée contenu
Synthèse	Optimisation	Réduction de dimension par ACP (250 → 50)	94 % de variance conservée, reco stables	Optimisation pertinente pour déploiement serverless
Synthèse	Architecture logicielle	Module Python + API Azure Functions	Logique découplée et réutilisable	Passage réussi vers une brique industrialisable
Synthèse	Démonstration applicative	Client Streamlit consommant l'API	Démo fonctionnelle et lisible	Séparation claire UI / moteur ML
Évolution	Données utilisateurs	Historique limité par utilisateur	Personnalisation partielle	Enrichissement des interactions nécessaire
Évolution	Qualité des recommandations	Similarité purement basée contenu	Résultats cohérents mais génériques	Ajout de signaux contextuels possible
Évolution	Scalabilité	Serverless Azure Functions	Scalabilité automatique	Architecture compatible montée en charge
Projection	Perspective produit	Architecture orientée services	Prête pour nouveaux utilisateurs / articles	Base saine pour évolution vers un produit réel

MERCI DE VOTRE ECOUTE

PROJET 10